

Implementing k-Nearest Neighbors (k-NN) from Scratch

Machine Learning

Assignment: Implementing the k-Nearest Neighbors (k-NN) Algorithm

Objective: The goal is to implement the k-NN classification algorithm from scratch, understand its instance-based nature, and analyze the impact of the hyperparameter 'k'.

Dataset Link: <https://archive.ics.uci.edu/dataset/53/iris>

Part (a): Implementation of the k-NN Model [50]

1. Understanding the k-NN Algorithm: k-NN is a non-parametric, lazy learning algorithm used for classification. To classify a new data point, it follows these steps:

1. **Calculate Distance:** Compute the distance between the new point and all points in the training data. You will use the **Euclidean distance**:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

2. **Find Neighbors:** Identify the 'k' nearest neighbors (the training data points with the 'k' smallest distances).
3. **Majority Vote:** Assign the class label to the new point based on the most frequent class among its 'k' neighbors.

2. Dataset Preparation: Use the **Iris dataset**. For this assignment, you will use **all 4 features** and **all 3 classes** (Setosa, Versicolor, Virginica).

3. Data Preprocessing: k-NN is highly sensitive to the scale of features because it is a distance-based algorithm. It is **essential** to **standardize** all features to have a mean of 0 and a variance of 1.

4. Task: Implement the k-NN Model: Implement a KNearestNeighbors class with the following methods:

- `__init__(k)`: To initialize the model with the number of neighbors, k.
- `fit(X_train, y_train)`: The fit method for k-NN is simple; it just needs to store the training data.
- `predict(X_test)`: This method will contain the core logic. For each sample in X_test, it must compute distances to all training samples, find the 'k' nearest neighbors, and return the predicted class based on a majority vote.

5. Training the Model: Split the preprocessed Iris dataset into a training set (80%) and a testing set (20%). Use the fit method to prepare your model.

Part (b): Evaluation and Visualization of the k-NN Model [50]

1. Hyperparameter Tuning for 'k': The choice of 'k' is critical to the model's performance. Train and evaluate your k-NN model for a range of different 'k' values (e.g., **k = 1, 3, 5, 7, 11, 15**). For each value of 'k', calculate and report the model's **accuracy** on the test set. Present these results in a clear table.

2. Visualization:

- Create a plot of **Accuracy vs. k-Value**. This plot will help you determine the optimal value of 'k' for this dataset and visualize the trade-off between bias and variance.
- To visualize the model's decision-making process, create a **decision boundary plot**. Since the full dataset has 4 features, you cannot plot it directly. For this task only, select any **two features** (e.g., Petal Length and Petal Width), retrain your model on this 2D data, and plot the decision boundaries for two different cases:
 - A low value of 'k' (e.g., $k=1$), which shows high variance.
 - A higher value of 'k' (e.g., $k=15$), which shows a smoother, high-bias boundary.

3. Analysis and Reporting: Write a brief report describing:

- Your implementation of the k-NN algorithm.
- An analysis of the "Accuracy vs. k-Value" plot. Explain how the model's performance changes as 'k' increases and justify your choice for the "best" 'k'.
- Discuss the **bias-variance trade-off** in the context of k-NN. Explain why a small 'k' leads to overfitting (low bias, high variance) and a large 'k' leads to underfitting (high bias, low variance).
- Use your decision boundary plots to visually explain the difference between a low-k and a high-k model.

Deliverables for Part (b): Submit the plots (Accuracy vs. k and decision boundaries), the evaluation results table, and the report.

Bonus Challenge:

- Implement a different distance metric, such as **Manhattan Distance**, and add a parameter to your class to switch between Euclidean and Manhattan. Compare the model's accuracy using both metrics.
- Implement **weighted k-NN**. In this version, the vote of each neighbor is weighted by the inverse of its distance ($\text{weight} = 1/\text{distance}$). This gives closer neighbors more influence on the final prediction. Compare the performance of your weighted k-NN against the standard version.
- **Note:** Do not use in-built functions for the core model logic (e.g., `sklearn.neighbors.KNeighborsClassifier`). You are expected to implement the distance calculation and voting mechanism yourself.